#1

Happy Supplies
Parts Warehouse

Customer Name: Jeff Peterson
Customer Number: HG54587
Customer Type: Consumer
Date: 7/1/2024
Time: 10:30am
Employee: D. Harrison

Part Number	Name	Type	Cage Code	Quantity Ordered	Unit Price
10654	Float Control	Plumbing	G413	4	12
10456	Modulator	Electrical	H433	3	7
10776	Hose Assembly	Plumbing	G413	7	9
10657	Float Assembly	Plumbing	G413	5	10

1NF (Has primary key and only atomic values)

order

partNumber	**customerNumber**	**date**	**time**	customerName	customerType	employee	name	type	cageCode	quantityOrdered	unitPrice
10654	HG54587	7/1/2024	10:30am	Jeff Peterson	Consumer	D. Harrison	Float Control	Plumbing	G413	4	12
10456	HG54587	7/1/2024	10:30am	Jeff Peterson	Consumer	D. Harrison	Modulator	Electrical	H433	3	7
10776	HG54587	7/1/2024	10:30am	Jeff Peterson	Consumer	D. Harrison	Hose Assembly	Plumbing	G413	7	9
10657	HG54587	7/1/2024	10:30am	Jeff Peterson	Consumer	D. Harrison	Float Assembly	Plumbing	G413	5	10

Table is in 1NF as ther primary key has beenidentified as consisting of partNumber, customerNumber, date, and time. Also, there are no repeting fields, and no repeating values in a record.

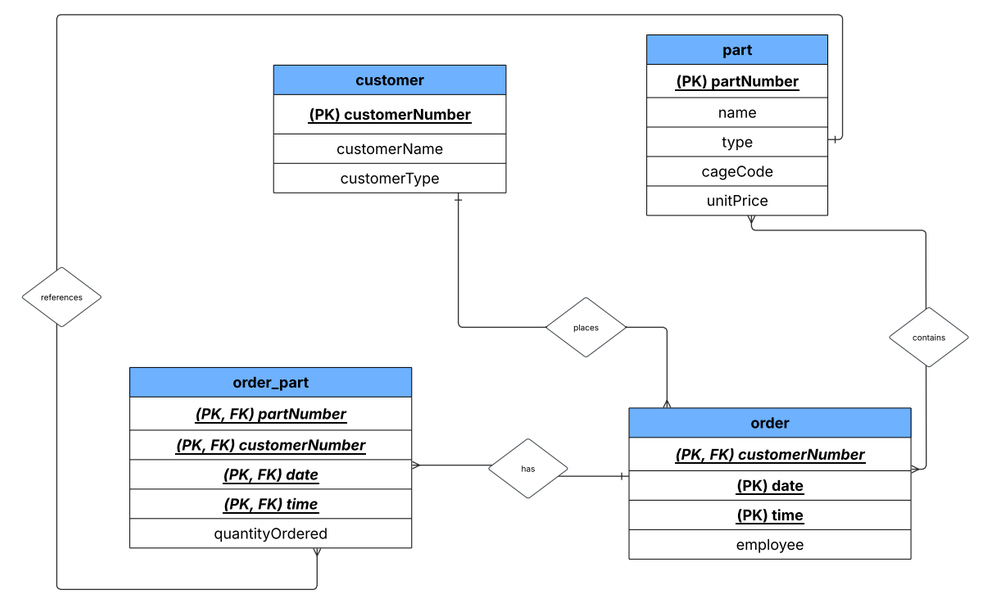
order (1NF)
(PK) partNumber
(PK) customerNumber
(PK) date
(PK) time
customerName
customerType
employee
name
type
cageCode
quantityOrdered
unitPrice

Composite PK = (partNumber, customerNumber, date, time)
All fields are atomic? -- Yes
So it passes 1NF

2NF (Remove partial dependencies)

The table transitions into 2NF when the table is split up to remove partial dependencies. Non key attributes relating to the customer, including customerType and customerName are only functionally dpeendent on customerNumber, so those attributes separate and become their own table. Non-key attributes relating to part information including name, type, cageCode, and unitPrice are only functionally dependent on the partNumber attribute in the primary key. So, those 5 attributes become their own table. the remaining non-key attributes, date, time and employee are only functionally dependent on customerNumber, beocming their own table. FInally, the order table, and part table have a many to many relationship, so a bridging table must be made so that each record, references a specific order of a specific part, by a specific customer, with a specific quantity of the part ordered.

Assumption:
- cageCode dpeneds direclty on partNumber



3NF (Remove transitive dependencies)
- None of the non-key attributes are functionally dependent on other non key attributes in each table, so it is already in 3NF.

These tables don't need to be modified further, they are already in 3NF as well, as there are no transitive dependences in any of the tables. each non-key attribute has a direct dependency on the primary key of its table. As shown by the cardinalities between the tables, a customer can place multiple orders, and order can have orders of multiple different parts, and a part can be present in multiple orders.

#2

Assumption:
- Date and Time are one attribute, and are atomic values when not spearated, considering the directions instruct us to not add any additional attributes.

appointment_info

staffNo	**apptDateAndTime**	patNo	patName	therapistName	branchNo
S1011	9/12/2022 10:00	P100	Lily White	Fred Smith	M15
S1011	9/12/2022 12:00	P105	Jill Baker	Fred Smith	M15
S1024	9/12/2022 10:00	P108	Andy McKee	Heidi Pierce	Q10
S1024	9/14/2022 14:00	P108	Andy McKee	Heidi Pierce	Q10
S1032	9/14/2022 16:30	P105	Jill Baker	Richard Levin	M15
S1032	9/15/2022 18:00	P110	Jimmy Winter	Richard Levin	B13

The composite Primary key is present (staffNo and apptDateAndTime), and the values in the table are atomic values. No repeating attributes, and no repeating values in one attribute of a record.

appointment_info (1NF)
(PK) staffNo
(PK) apptDateAndTime
patNo
patName
therapistName
branchNo

1NF (has composite primary key, atomic values in fields, and no repeating values

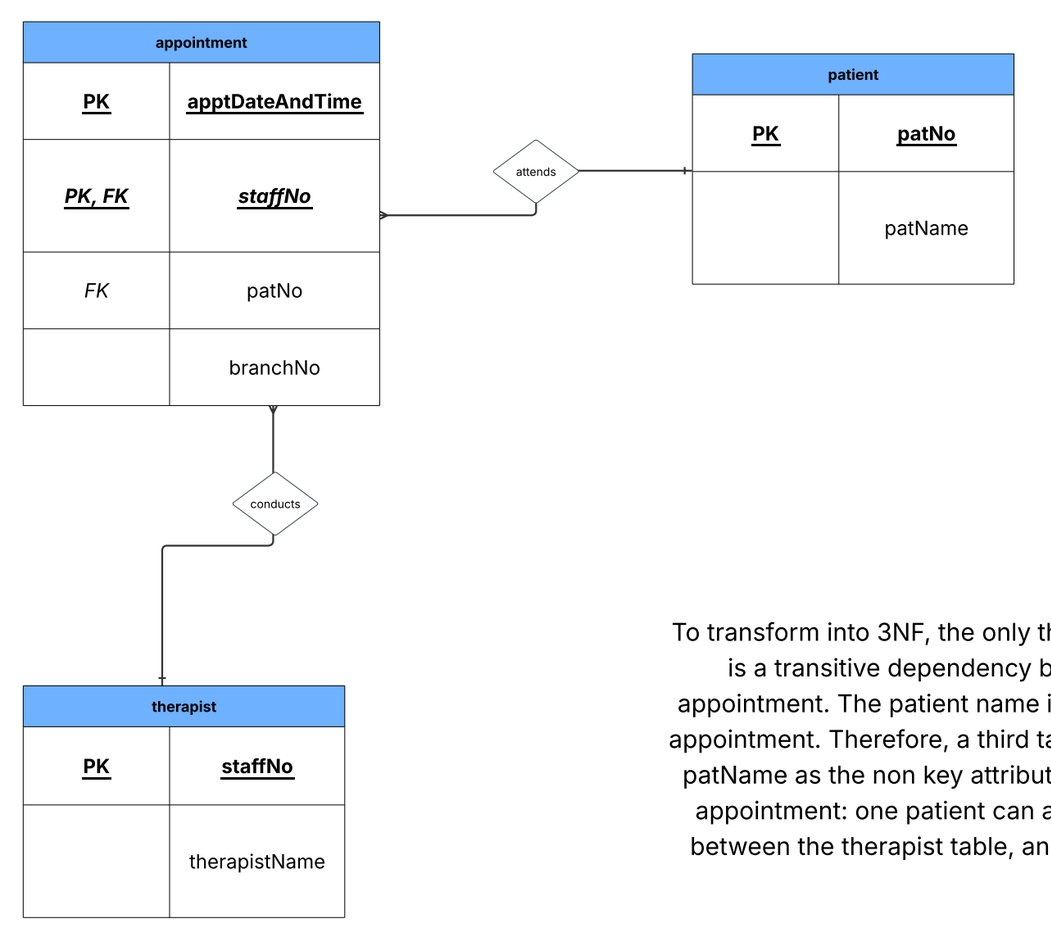
patient_appointment

PK	**apptDateAndTime**
PK, FK	**staffNo**
	patNo
	patName
	branchNo

therapist

PK	**staffNo**
	therapistName

The table transitions into 2NF when the table is split up to remove partial dependencies. Non key attributes relating to the patient, including patNo and patName, and branchNo are functionally dependent on both apptDateAndTime and staffNo, since a therapist can be only in one locatio branch per day. So those attributes separate and become their own table. The non-key attribute therapistName in the therapist table is only functionally dependent on the staffNo attribute of the composite primary key.



To transform into 3NF, the only thing that would need to be split apart would be patName, since there is a transitive dependency between patName and the composite primary key -- the specific appointment. The patient name is first determined by the patNo, which is determined by the specific appointment. Therefore, a third table is created titled paient, containing patNo as the primary key, and patName as the non key attribute. That way, there is a one to many relationship betwrrn patient and appointment: one patient can attend multiple therapy appointments. A one to many relation exists between the therapist table, and appointment table, as one therapist can conduct multiple therapy appointments.

#3

1NF (has composite primary key, atomic values in fields, and no repeating values)

maid_better

eNo	**contractNo**	eventNo	hours	eName	eventLoc
1135	C1024	H25	16	Smith J	Queens
1057	C1024	H25	24	Hocine D	Queens
1068	C1025	H4	28	White T	Yonkers
1135	C1025	H4	15	Smith J	Yonkers
1135	C1026	H25	10	Smith J	Queens

Assumption:
- eventLoc only directly depends on eventNo

maid_better (1NF)
(PK) eNo
(PK) contractNo
eventNo
hours
eName
eventLoc

The composite Primary key is present (eNo and contractNo), and the values in the table are atomic values. No repeating attributes, and no repeating values in one attribute of a record.

2NF (partial dependencies have been separated into separate tables)

employeer_hour

PK, FK	***eNo***
PK, FK	***contractNo***
	hours

contract

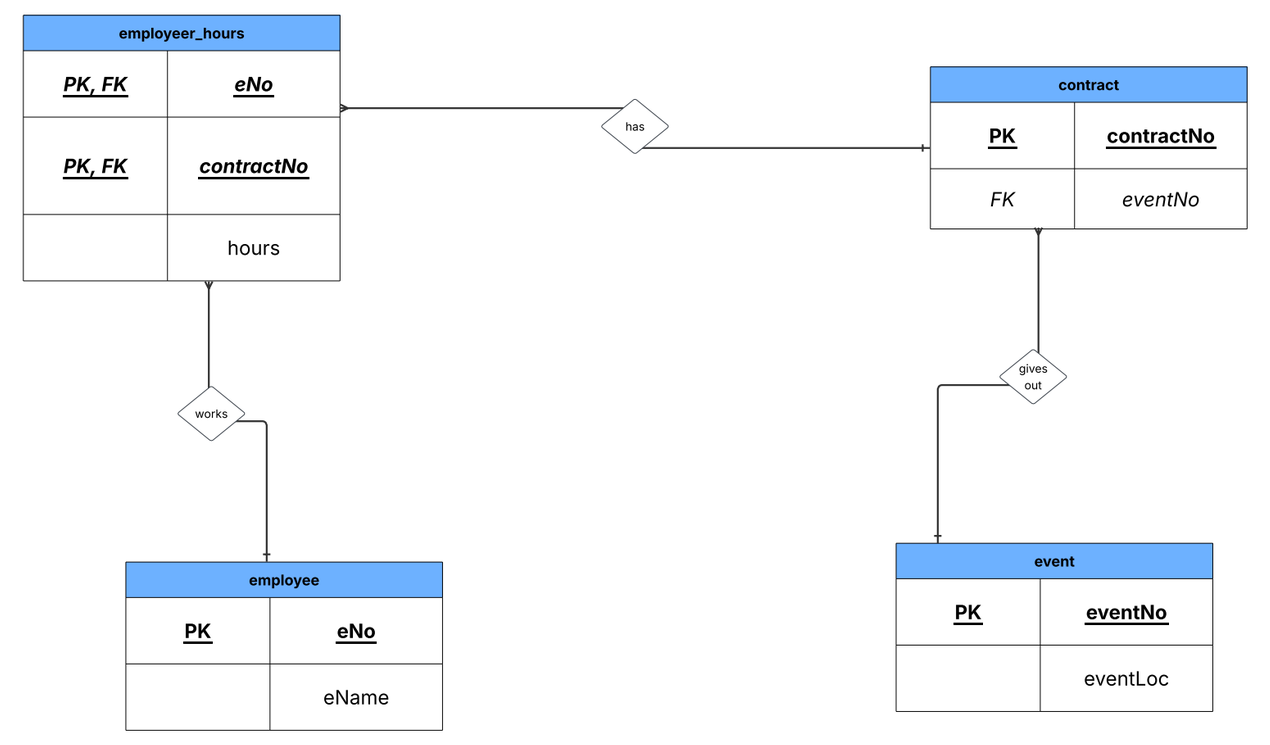
PK	**contractNo**
	eventNo
	eventLoc

employee

PK	**eNo**
	eName

The table transitions into 2NF when partial dependencies are found between eventNo, eventLoc, and contractNo, as the contractNo determines the eventNo and eventLoc. eName only depends on the eNo, and so a separate table is made for that, making eNo the PK for that table, contractNO, the PK for the contract table, and eNO and contractNo as both PK and FK in the employee_hour table.

3NF (transitive dependencies have been separated int oa different table)



The table transitions into 3NF when a transitive dependency is found between eventLoc, eventNo, and contractNo. The eventLoc depends on eventNo, but eventNo is only determined by contractNo. So, the eventNo becomes a PK in its own table, and a FK in the contract table. One event can assign multiple contracts, one contract can have multiple employees working on it, and one employee can work multiple contracts.